College Football Bowl Game Predictor

UE24CS352A Machine Learning - Mini-Project Write-up | Team: [Member 1 name, SRN], [Member 2 name, SRN] | Problem No.: [#]

1. Problem Statement

Each bowl season, fans play a Pick'em game: pick the winner of every bowl and assign each pick a unique confidence from 1 to N (N = number of bowls); a correct pick earns its confidence points. Winning needs both correct winners and well-placed confidence. We predict, from regular-season statistics of the two teams, the **score margin** of each bowl. The sign gives the winner and the magnitude gives the confidence. This is therefore a **regression** problem (a re-implementation of the Stanford CS229 project by Cheshire, Childs and Leung).

2. Dataset

The reference project used 2000-2010 college-football box scores (about 350 bowl games, 36 features per team). We use 12 per-team features (season averages/variances): offensive points, first downs, completion %, yards per carry, kick-return yards, penalty yards; defensive points allowed, turnovers, punt-return yards allowed, FG % allowed, penalty yards and variance of touchdowns allowed. **Important:** the original dataset was not accessible when this report was produced, so the results below come from a **synthetic stand-in dataset with the same schema** (352 games, 11 seasons), generated from latent team strengths. The pipeline reads `data/bowl_games.csv` when present; all numbers should be regenerated on the real data before final submission.

3. Approach

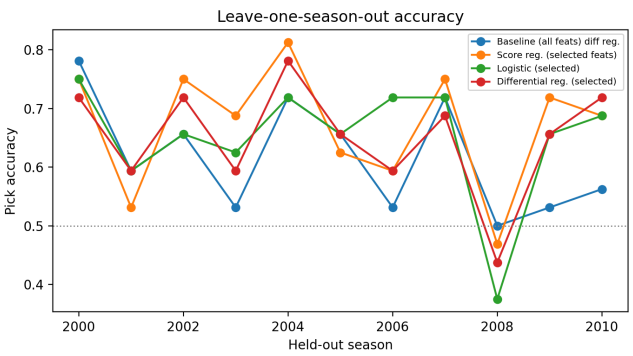
Score regression: linear regression of a team's points on its own offence and the opponent's defence; predicted margin = difference of the two predicted scores. **Differential regression:** linear regression of margin directly on both teams' features. **Feature selection:** for each of 11 leave-one-season-out OLS fits we obtain p-values; a feature is kept in a run if min p < 0.1 and max p < 0.4. Because Team 1/Team 2 order is arbitrary, this is repeated over 100 random orderings and we keep features that pass in at least 20% of runs for *both* teams. **Comparison models:** logistic regression (winner) and SVM regression (margin). **Evaluation:** leave-one-season-out CV (train on 10 seasons, test on the held-out bowl season): pick accuracy, margin MSE, and simulated Pick'em score using |predicted margin| as the confidence ranking.

4. Implementation Overview

Python (NumPy, pandas, SciPy, scikit-learn, Matplotlib). `football_predictor.py` contains data loading, models, p-value feature selection, CV and plotting; `football_predictor.ipynb` walks through the same steps; a README gives setup and run instructions. The code is in a private GitHub repository shared with the faculty/TAs.

5. Results

Model	Accuracy	Margin MSE	Pick'em % of max
Baseline (all feats) score reg.	65.6%	128.8	70.2%
Baseline (all feats) diff reg.	61.6%	139.6	66.2%
Score reg. (selected feats)	67.0%	127.3	70.9%
SVM regression (selected)	58.8%	142.0	64.3%
Logistic (selected)	65.1%	143.1	70.8%
Differential reg. (selected)	65.1%	130.1	70.7%



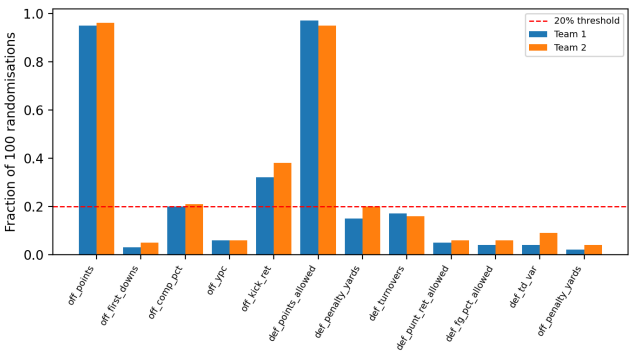


Figure 1: Per-season accuracy (left). Figure 2: Feature-selection frequency over 100 randomisations (right).

6. Conclusions

Margin-based regression gives a winner and a confidence from a single model. On the stand-in data the selected-feature models reach about 65-67% accuracy against 50% for coin-flip picking, and simulated Pick'em scores of about 71% of

the maximum versus about 49% for random picks, so feature selection and margin-based confidence ranking add real lift over the all-features baseline. SVM regression did not beat linear regression without tuning. Per-season accuracy varies widely (roughly 44% to 78%), mirroring the upset-driven variance noted in the original work. Because the data is synthetic these figures validate the pipeline, not real football performance; the original project reported about 66% on real data (Cortana 67%, Elo 70%). Future work: real data, PCA, tuned SVM, and an upset-detection model.